Lecture 06: GUI Programming & Threads

Objectives:

- Learn how to write GUI Programs.
- Learn how to write Multi-threaded Programs.

1. GUI Programming

In C#, GUI applications are created by extending the **System.Forms.Form** class.

Writing GUI involves two things, namely, designing the user-interface, and event-handling.

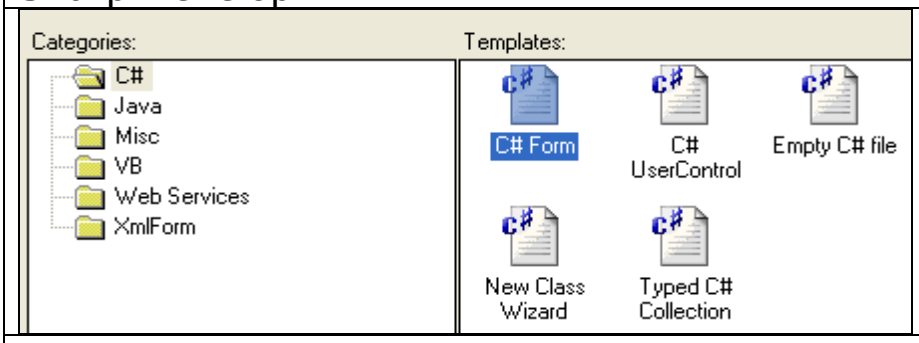
Both these aspects have been simplified by the development environments (Visual Studio and SharpDevelop as described below:

A. Designing the user interface

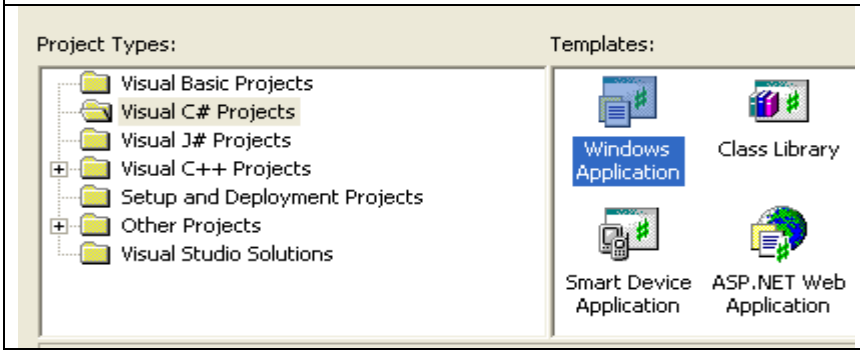
To design a user interface using Visual Studio or SharpDevelop, follow the following steps:

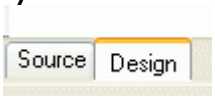
1. Specify that you are creating a **Windows Application** (**C# Form** in SharpDevelop) when creating a new project or file.

Sharp Develop



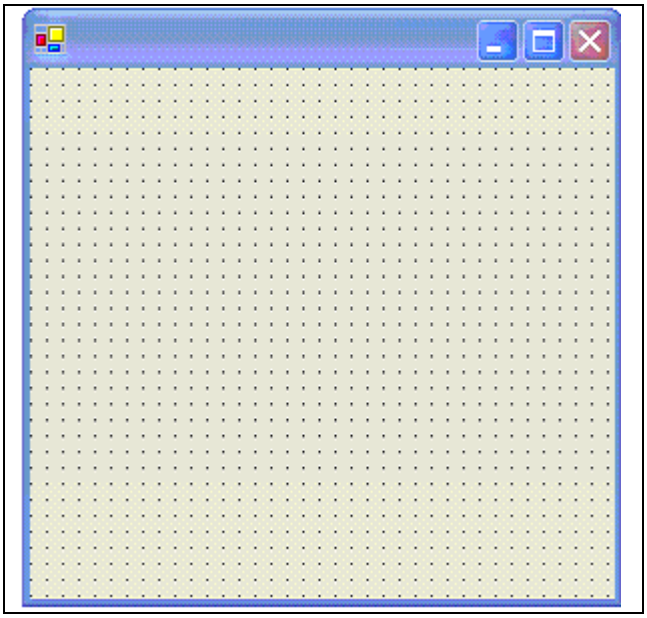
Visual Studio



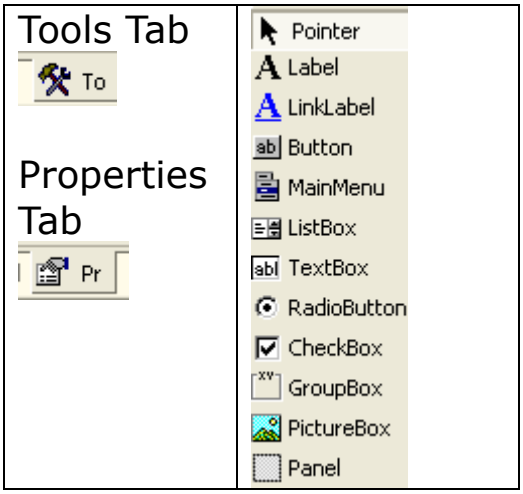
This will allow you to have two views of your program, **Source** and **Design.** 

The design view allows you to create a user interface by simply dragging the controls from the tools window into your design window

2. Switch to the Design tap. This will open a blank design-window:



3. Open the Tools Window. This will allow you to drag the various controls you desire to the form design window and organize them to suit your application.



4. Use the Properties Window to change the behavior and appearance of the controls.

Some of the properties that are commonly changed include:

- The variable **names** automatically generated for the controls
- The **Text** property
- Multi-lines for TextBox (A text box is just a multi-line text field)
- Scrollers for multi-line tools
- Fonts (size, color, appearance, etc.)

The following is a sample user-interface that can be easily generated following the above steps.



Once you finish the design, switch to the source code to see the code that will be generated.

Example 1: The following is a sample code automatically generated for the above design.

```
using System;
using System.Windows.Forms;

namespace WindowsApp {
    public class BreakURL : System.Windows.Forms.Form
    {
        private System.Windows.Forms.TextBox resultBox;
        private System.Windows.Forms.TextBox addressBox;
        private System.Windows.Forms.Button button;
        private System.Windows.Forms.Label label;
        public BreakURL()
        {
            InitializeComponent();
```

```
        }

        void InitializeComponent() {
            this.label = new System.Windows.Forms.Label();
            this.button = new
System.Windows.Forms.Button();
            this.addressBox = new
System.Windows.Forms.TextBox();
            this.resultBox = new
System.Windows.Forms.TextBox();
            this.SuspendLayout();
            //
            // label
            //
            this.label.Font = new
System.Drawing.Font("Microsoft Sans Serif", 9.75F,
System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((System.Byte)(0)));
            this.label.ForeColor =
System.Drawing.SystemColors.ActiveCaption;
            this.label.Location = new
System.Drawing.Point(24, 16);
            this.label.Name = "label";
            this.label.Size = new System.Drawing.Size(200,
16);
            this.label.TabIndex = 0;
            this.label.Text = "Enter address to break";
            //
            // button
            //
            this.button.Location = new
System.Drawing.Point(328, 40);
            this.button.Name = "button";
            this.button.Size = new System.Drawing.Size(88,
23);
            this.button.TabIndex = 3;
            this.button.Text = "Break Address";
            //
            // addressBox
            //
            this.addressBox.Location = new
System.Drawing.Point(24, 40);
            this.addressBox.Name = "addressBox";
            this.addressBox.Size = new
System.Drawing.Size(280, 20);
```

```
            this.addressBox.TabIndex = 1;
            this.addressBox.Text = "";
            //
            // resultBox
            //
            this.resultBox.BackColor =
System.Drawing.Color.White;
            this.resultBox.Location = new
System.Drawing.Point(24, 72);
            this.resultBox.Multiline = true;
            this.resultBox.Name = "resultBox";
            this.resultBox.ScrollBars =
System.Windows.Forms.ScrollBars.Both;
            this.resultBox.Size = new
System.Drawing.Size(280, 80);
            this.resultBox.TabIndex = 2;
            this.resultBox.Text = "";
            //
            // CreatedForm
            //
            this.AutoScaleBaseSize = new
System.Drawing.Size(5, 13);
            this.ClientSize = new System.Drawing.Size(432,
182);
            this.Controls.AddRange(new
System.Windows.Forms.Control[] {
                                this.button,
                                this.resultBox,
                                this.addressBox,
                                this.label});
            this.Name = "CreatedForm";
            this.ResumeLayout(false);
        }
        public void BreakAddress(Object source, EventArgs
arg) {

        }

        public static void Main() {
            Application.Run(new BreakURL());
        }
    }
}
```

Important Note: The code generated by the IDE inside the method, **InitializeComponent**() should not be changed manually if you wish to go back to the design window.

This is because the IDE will override any manual changes you made if you go back to the design window. To make modifications, use the design window. Any additional code should be added inside the constructor (after the call to the InitializeComponet) or somewhere else.

B. Handling Events

In C#, events are handled using event delegates. Each control tool has a number of event delegates of type, **EventHandler**, to which you can register your event handling methods. Some of the predefined EventHandler delegates that many of the controls have are:

Click
Closed
Closing
Leave
etc.

As discussed earlier, the EventHander delegate expects methods with signature as follows:

```
void MethodName(Object source,  EventArgs arg)
```

To register for events, select the events tab,  , from within the properties window. This will list all the events that the current control has.

For each event that you wish to register, type the method name that you wish to be executed when the event occurs. If the method does not exists, the system will automatically create it with an empty body, which you can then fill.

In the example above, an empty method:

```
void BreakAddress(Object source, EventArgs arg)
```

is created and registered with the **Click** method of the button control.

The final thing you need to do for your program to execute is to create a main method. In the main method, the static **Run** method

of the **Application** class is called, passing to it, an instance of the form as argument.

2. Writing Multi-threaded Applications

In C#, multi-threaded applications are created by creating an instance of **System.Threading.Thread** class.

The constructor of the **Thread** class takes an instance of **System.Threading.ThreadStart** delegate as parameter.

The ThreadStart delegate in turn takes a method of the form:

void MethodName()

Thus, the first thing to do is to enclose the code that is to be executed by a thread, into a method of the above signature.

An Instance of **Thread** is then created and associated with the method as described above.

Once an instance of **Thread** is created, it can be started by calling its **Start()** method.

Example 2: The following example demonstrates these steps

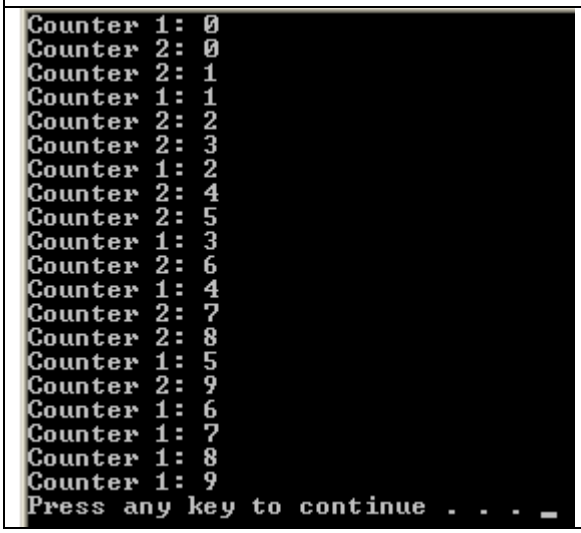
```
using System;
using System.Threading;

public class ThreadedCounters {
    public static void Main(){
        Thread thread1 = new Thread(new
ThreadStart(Counter1));
        thread1.Start();

        Thread thread2 = new Thread(new
ThreadStart(Counter2));
        thread2.Start();
    }

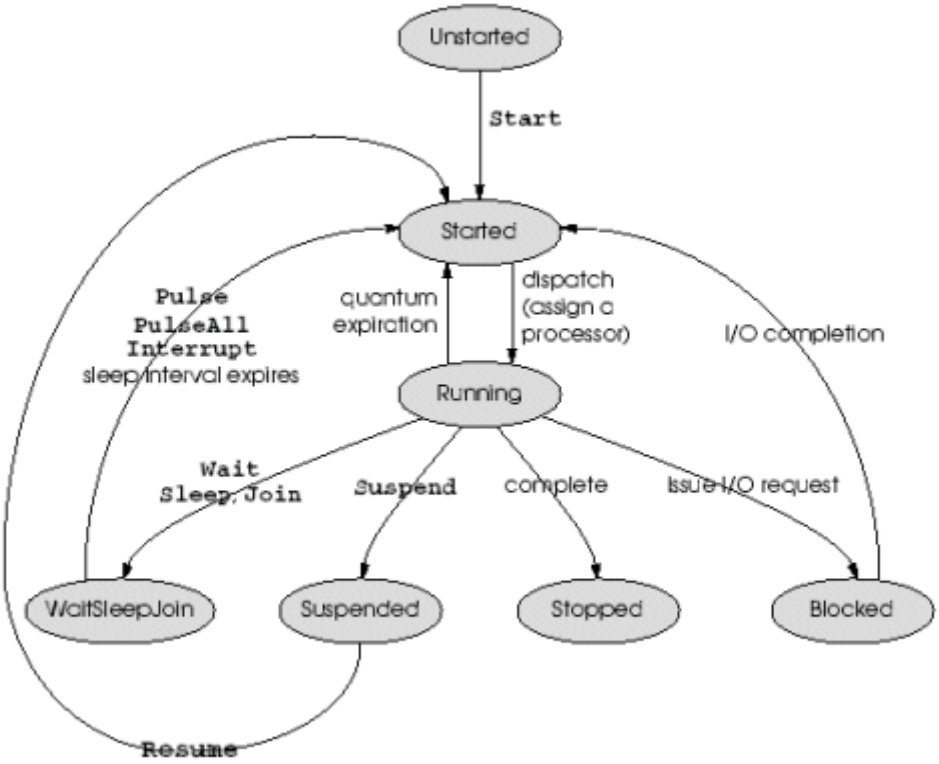
    public static void Counter1() {
        for (int i = 0; i<10; i++) {
            Console.WriteLine("Counter 1: "+i);
            Thread.Sleep(35);
        }
    }
```

```
public static void Counter2() {
          for (int i = 0; i<10; i++) {
               Console.WriteLine("Counter 2: "+i);
               Thread.Sleep(20);
          }
     }
 }
```



LifeTime Cycle of a Thread:

A Thread object can have many possible states as shown by the following figure and table. The possible states for a Thread object are members of the **System.Threading.ThreadState** enumeration summarized below:



Member	Description
Aborted	The thread is in the Stopped state.
AbortRequested	**ThreadAbort()** method has been invoked on the thread, but the thread has not yet received the **System.Threading.ThreadAbortException** that will try to stop it.
Background	The thread is being executed as a background thread, as opposed to a foreground thread. This state is controlled by setting the **IsBackground** property of the Thread class.
Running	The thread has been started, it is not blocked, and there is no pending **ThreadAbortException**.
Stopped	The thread has stopped.
StopRequested	The thread is being requested to stop. This is for internal use only.
Suspended	The thread has been suspended.
SuspendRequested	The thread is being requested to suspend.
Unstarted	The **Thread.Start** method has not been invoked on the thread.

WaitSleepJoin	The thread is blocked as a result of a call to **Wait**, **Sleep** or **Join** methods.

When first created, a thread is in the **Unstarted** state. After the *Start* method is called, the thread's state will enter the **Started** state. When it gets a processor, it enters the **Running** state.

A thread can be put in **Suspended** state by calling the *Suspend* method. A suspended thread is resumed when the *Resume* method is called.

The *Sleep* method is often used, as the name implies, to send a thread to sleep for a specified period of time. After the time elapses, the thread wakes up and continues running automatically.

A Thread object should be terminated when the application exits. To do this, you use the ***Abort*** method.
The Thread class has the ***IsAlive*** property that can be used to inquire about the state of a Thread object. If ***IsAlive*** property is True, the thread has been started and has not been aborted.

A thread can also run in background or in foreground. A background thread is the same as a foreground thread, except that background threads do not prevent a process from terminating.

A Thread is put in background state by setting the Thread class's **IsBackground** property to True.

Example 3:
The following example shows how to use threads to write GUI applications which are responsive.

Without using thread, a GUI control could be inactive while some computation is going on, which is an undesirable behavior.

```
using System;
using System.Windows.Forms;
using System.Drawing;
using System.Threading;

public class SalamShabab : System.Windows.Forms.Form
{
    private System.Windows.Forms.Label salam;
```

```
    private System.Windows.Forms.Button button;
    private int upperY, lowerY, salamX, salamY;
    private Thread salamThread;
    private bool start;

    public SalamShabab()
    {
        InitializeComponent();
        salamY = upperY = salam.Location.Y;
        lowerY = this.Bounds.Bottom - 40;
        salamX = salam.Location.X;
        salamThread = new Thread(new
ThreadStart(MoveSalam));
        button.Click += new EventHandler(StartAnimation);
        this.Closed += new EventHandler(StopThreads);
    }

    void InitializeComponent() {
            this.button = new
System.Windows.Forms.Button();
            this.salam = new System.Windows.Forms.Label();
            this.SuspendLayout();
            //
            // button
            //
            this.button.Font = new
System.Drawing.Font("Microsoft Sans Serif", 8.25F,
System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((System.Byte)(0)));
            this.button.Location = new
System.Drawing.Point(104, 8);
            this.button.Name = "button";
            this.button.TabIndex = 2;
            this.button.Text = "Start";
            //
            // salam
            //
            this.salam.Font = new
System.Drawing.Font("Microsoft Sans Serif", 12F,
System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((System.Byte)(0)));
            this.salam.ForeColor =
System.Drawing.Color.Green;
            this.salam.Location = new
System.Drawing.Point(72, 40);
```

```
            this.salam.Name = "salam";
            this.salam.Size = new System.Drawing.Size(168,
23);
            this.salam.TabIndex = 0;
            this.salam.Text = "Salaam Shabab";
            //
            // CreatedForm
            //
            this.AutoScaleBaseSize = new
System.Drawing.Size(5, 13);
            this.ClientSize = new System.Drawing.Size(292,
266);
            this.Controls.AddRange(new
System.Windows.Forms.Control[] {
                        this.button,
                        this.salam});
            this.Name = "CreatedForm";
            this.ResumeLayout(false);
        }

        public void MoveSalam() {
            while (true) {
                while (salamY <= lowerY) {
                    salamY += 10;
                    salam.Location = new Point(salamX,
salamY);
                    Refresh();
                    Thread.Sleep(20);
                }
                while (salamY >=upperY) {
                    salamY -= 10;
                    salam.Location = new Point(salamX,
salamY);
                    Refresh();
                    Thread.Sleep(20);
                }
            }
        }
        public void StartAnimation(Object source, EventArgs
args) {
            if (!start) {
                start = true;
                button.Text = "Stop";
                if (salamThread.IsAlive)
                    salamThread.Resume();
```

```
                    else
                        salamThread.Start();
                }
                else {
                    start = false;
                    button.Text = "Start";
                    salamThread.Suspend();
                }
            }

            public static void Main() {
                Application.Run(new SalamShabab());
            }

            public void StopThreads(Object source, EventArgs
args) {
                try {
                    if (salamThread.IsAlive)
                        salamThread.Abort();
                }
                catch(Exception) {
                }
            }
}
```

Thread Scheduling & Synchronization:

A common problem that needs to be handled when writing multi-threaded program is thread synchronization.

This is necessary where more than one thread needs to modify a certain object at a time.

Example 4: Consider the Unsafe banking example below:

```
using System;
using System.Threading;

public class BankAccount {
     int balance = 0;

     public BankAccount(int initial) {
          balance = initial;
     }

     public void Deposit(int amount) {
          balance+=amount;
     }

     public void Withdraw(int amount) {
          balance-=amount;
     }

     public int GetBalance() {
          return balance;
     }
}


public class UnsafeBanking {
     static Random randomizer = new Random();
     static BankAccount account = new BankAccount(100);

     public static void Main() {

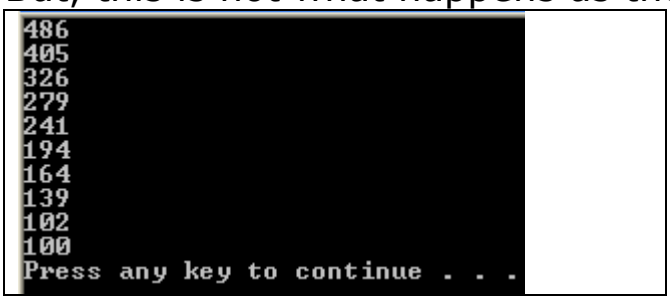
          Thread[] banker = new Thread[10];
          for (int i=0; i<10; i++) {
               banker[i] = new Thread(new
ThreadStart(DepositWithdraw));
               banker[i].Start();
          }
```

```
    }

    public static void DepositWithdraw() {
        int amount = randomizer.Next(100);
        account.Deposit(amount);
        Thread.Sleep(100);
        account.Withdraw(amount);
        Console.WriteLine(account.GetBalance());
    }
}
```

From the code, since the amount being deposited is the same as the amount withdrawn, one would expect the balance to remain unchanged.

But, this is not what happens as the following output shows:



To solve this problem, C# provides a number of options as discussed below:

Using the *Monitor* Class:

The monitor class provides two static methods, ***Enter*** and ***Exit***.

The ***Enter*** method is used to obtain a lock on an object that the monitor guards and is called before accessing the object.
If the lock is currently owned by another thread, the thread that calls Enter blocks—that is, is taken off the processor and placed in a very efficient wait state—until the lock becomes free.

Exit frees the lock after the access is complete so that other threads can access the resource.

Example 5: The following uses Enter/Exit methods to synchronize access to the account object:

```
public static void DepositWithdraw() {
     int amount = randomizer.Next(100);
     Monitor.Enter(account);
     try {
          account.Deposit(amount);
          Thread.Sleep(100);
          account.Withdraw(amount);

Console.WriteLine(account.GetBalance());
     }
     finally {
          Monitor.Exit(account);
     }
}
```



Note that calls to ***Exit*** are enclosed in finally blocks to ensure that they're executed even when there are exceptions. Always use finally blocks to exit monitors or else you run the risk of causing other threads to hang indefinitely.

Using the *lock* keyword:

An alternative to using the Enter/Exit method is to use the lock keyword around the code that should be accessed by one thread as shown below:

```
public static void DepositWithdraw() {
     int amount = randomizer.Next(100);
     lock(account) {
          account.Deposit(amount);
          Thread.Sleep(100);
          account.Withdraw(amount);

     Console.WriteLine(account.GetBalance());
     }
}
```



Synchronizing Access to Collections

The collections classes in the .NET Framework class library are not thread-safe. For example, If you want to share an **ArrayList** between a reader thread and a writer thread, it's important to synchronize access to the ArrayList so that one thread can't read from it while another thread writes to it.

One way to synchronize access to an ArrayList is to use a monitor or lock, as demonstrated below:

```
ArrayList list = new ArrayList ();
// Thread A
lock (list) {
    list.Add ("Fender Stratocaster");
}
// Thread B
lock (list) {
    string item = (string) list[list.Count - 1];
}
```

However, there's an easier way. ArrayList, Hashtable, Queue, Stack, and selected other standard classes implement a method named ***Synchronized*** that returns a thread-safe wrapper around the object passed to it.

```
// Create the ArrayList and a thread-safe wrapper for it
ArrayList list = new ArrayList ();
ArrayList safelist = ArrayList.Synchronized (list);

// Thread A
safelist.Add ("Some Item");

// Thread B
string item = (string) safelist[safelist.Count - 1];
```

Using thread-safe wrappers created with the Synchronized method shifts the burden of synchronization from your code to the framework.

It can also improve performance because a well-designed wrapper class can use its knowledge of the underlying class to lock only when necessary.